

```
a,b=1;main(c){printf("0\n");
while(c^832040)
c=a+b,a=b,b=c,printf("%d\n",c);}
```

The program *fib2.c* (78 characters) placed me at the 49th position among those who got the solutions. But I was amazed to learn that the current best solution has just 46 characters. Knowing that limit, I didn't try to improve my solution any further. In all likelihood, the best solution might have used recursion to solve this puzzle because generating the Fibonacci series is an inherently recursive problem. To further illustrate the fact that in code golfing the only requirement is to write a program with minimum source code, take a look at Figure 1, which shows the warnings during the compilation of *fib2.c* and the first few lines of the output.

Now let us go through the program *fib2.c* to understand the code golfing techniques used and the reason for the many warnings. First, the three variables used are named *a*, *b* and *c* — single character variable names. I am sure software companies following good programming practices will fire you immediately for such variable names but in code golfing, source code size is the only thing that matters. Further, we haven't declared the data type of these variables. Due to the *implicit int* rule of C, the three variables are of type integer, by default, even though you get a warning during compilation.

Since the variable *a* is declared as a global variable, it is initialised to 0 automatically. Thus we can save a few more characters. Notice that the variable *c* is declared inside the *main()* function because this function can also have arguments. This helps you save one more character, a comma separating multiple variables. The test condition of the *while* loop is *while(c^832040)* instead of *while(c != 832040)*, thus saving one more character. The four statements *c=a+b, a=b, b=c*



```
File Edit View Search Terminal Help
# gcc fib.c
fib.c:1:1: warning: data definition has no type or storage class
a,b=1;main(c){printf("0\n");while(c^832040)c=a+b,a=b,b=c,printf("%d\n",c);}
^
fib.c:1:1: warning: type defaults to 'int' in declaration of 'a' [-Wimplicit-int]
fib.c:1:3: warning: type defaults to 'int' in declaration of 'b' [-Wimplicit-int]
a,b=1;main(c){printf("0\n");while(c^832040)c=a+b,a=b,b=c,printf("%d\n",c);}
^
fib.c:1:7: warning: return type defaults to 'int' [-Wimplicit-int]
a,b=1;main(c){printf("0\n");while(c^832040)c=a+b,a=b,b=c,printf("%d\n",c);}
^~~~~~
fib.c: In function 'main':
fib.c:1:7: warning: type of 'c' defaults to 'int' [-Wimplicit-int]
fib.c:1:15: warning: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
a,b=1;main(c){printf("0\n");while(c^832040)c=a+b,a=b,b=c,printf("%d\n",c);}
^~~~~~
fib.c:1:15: warning: incompatible implicit declaration of built-in function 'printf'
fib.c:1:15: note: include '<stdio.h>' or provide a declaration of 'printf'
# ./a.out
0
1
1
2
3
5
8
13
21
```

Figure 1: The warnings and output of the program *fib2.c*

and *printf("%d\n",c)* are separated by commas, making it a single statement. Hence the *while* loop does not require opening and closing brackets. Also notice that I haven't included any header files, resulting in even more warnings during compilation. Thus, from the above example, it is clear that you need a clear understanding about the different features of the programming language you are using for code golfing. But to win code golfing competitions you also need to develop better algorithms to solve the given problem, and out-of-the-box thinking.

The website mentioned earlier, which offers code golfing puzzles, does not allow users to inspect winning solutions by other programmers — in spite of that the puzzles and the competition format of the website is quite interesting. But if you really want to inspect the solutions provided by other programmers for different code golfing challenges, I urge you to visit the Code Golf website of the Stack Exchange Network. The link <https://codegolf.stackexchange.com/> will take you there. You may not be familiar with this website, but I am sure you have all heard about Stack Overflow, also from the Stack Exchange Network.

Another important question that needs to be answered about code golfing is regarding the most

suitable programming language for participating in such challenges. Notice that the size of the source code often depends on the programming language being used. To understand this point better, consider a Python script and a C program that prints the message 'Hello World'. The Python script needs just one line whereas the C program needs additional constructs like the declaration of the *main()* function, opening and closing brackets, etc, other than the *printf()* statement. Due to this reason, separate code golfing competitions are often conducted for different programming languages. But on the rare occasion where the competition is programming language agnostic, which language is preferred? The choice definitely depends on the programming language you are most familiar with. I usually attempt solutions in C or Python, depending on the specific problem at hand. But let's suppose someone really wants to win code golfing competitions, to the extent of being willing to learn a new programming language. In that case, which programming language should be chosen? One choice is Perl, a general-purpose programming language that allows you to write very crisp programs. In fact, code golfing competitions were initially held by